

プロジェクト計画整理用シート

1. プロジェクト概要

本プロジェクトでは、13 週をかけて Arduino を用いた楽器システムを開発する。

最終的なゴールは、指揮動作に応じて複数の Arduino がリアルタイムに同期し、可変テンポで演奏および視覚表現を行うシステム作成する。

(※「何が動けば完成か」を一文で書くこと)

本プロジェクトでは、ハードウェアとソフトウェアを組み合わせたシステムとして動作することを重視し、第 11 週にはデモと発表を行う。

2. 成果物の定義(完成条件)

本プロジェクトの完成条件は以下の通りとする。

- 最低限達成すべき機能: センサーから値を取得できる。正しく同期ができる。演奏が完走する。マトリクス出力ができる。
- 可能であれば実現したい追加機能: 音のエフェクト。カエルの鳴き声。Arduino 間のマトリクス表示。
- 動作確認の方法(どのように動けば成功か): 指揮棒を振ってそれに応じて最初の音になり、4つの Arduino がそれぞれ演奏を開始してマトリクス表示を行う。また、途中で距離に応じてテンポを変える。

これらを満たしていれば、「完成した」と判断する。

3. 全体スケジュールの考え方

13 週を、以下のフェーズに分けて進める。

- 企画・要件整理フェーズ(第__1__週～第__2__週)
- 設計フェーズ(第__3__週～第__4__週)
- 計画発表準備フェーズ(第__4__週～第__5__週)
- 個別実装フェーズ(第__6__週～第__8__週)
- 統合・評価フェーズ(第__9__週～第__11__週)
- 発表準備フェーズ(第__11__週～第__12__週)
- 報告書準備フェーズ(第__12__週～第__13__週)

各フェーズの終わりには、「ここまでできていれば次に進める」という状態を明確にする。

4. 作業分解(WBS)

プロジェクトを進めるために、作業を以下のように分解する。

まず、企画・設計に関する作業として、本システムの目的および要求仕様を整理し、システム全体の構成、各 Arduino の役割分担、通信方式、同期方式、テンポ変化の処理方法を設計する。また、ハードウェア設計として回路構成およびセンサー選定を行い、ソフトウェア設計としてテンポ変更ロジックおよび同期フラグ生成ロジックを設計する。さらに、インターフェース設計として Arduino 間の通信方法および入出力仕様を整理し、指揮動作を入力するための指揮棒などの物理デバイスの設計を行う。

次に、ハードウェアに関する作業として、回路設計および回路図の作成を行い、センサーおよび Arduino を用いた回路の実装を行う。また、指揮動作を入力するためのインタフェースとして、指揮棒などの物理デバイスの製作を行う。

ソフトウェアに関する作業として、センサー取得モジュールおよび通信制御モジュールを実装する。センサー取得モジュールでは、回路からセンサー情報を取得し、データ整形および閾値設定を行った上で、同期フラグおよびテンポ変化フラグを生成する。通信制御モジュールでは、これらの情報を受信し、各 Arduino への割り当ておよび送信制御を行う。さらに、楽器システムでは、演奏開始フラグに応じた音の生成および出力処理を実装し、LED マトリクス表示を行う。また、テンポ変化フラグに応じて演奏テンポを動的に変更する機能を構築する。加えて、Processing を用いた音生成システムを構築し、Arduino から送信された演奏情報に基づいて音を生成・出力する。最後に、各モジュールの結合を行い、システム全体の統合を行う。

統合・テストに関する作業として、回路の動作確認としてシリアル出力によるセンサー値の確認を行い、センサー値の妥当性および閾値の適切性を検証する。また、加速度センサーおよび距離センサーの動作確認を行い、同期フラグおよびテンポ変化フラグが正しく生成されているかを確認する。さらに、各システムを統合し、複数デバイス間での同期動作、演奏開始フラグの反映、およびテンポ変化の反映が正しく行われるかを確認する。

各作業について、「何ができれば完了か」を明確にする。

5. 工数の見積もり

各作業に対して、おおよその作業時間を見積もる。

- 作業 A(回路設計): __ 4 __ 時間程度かかると想定
- 作業 B(センサー制御): __ 50 __ 時間程度かかると想定
- 作業 C(同期制御): __ 30 __ 時間程度かかると想定
- 作業 D(楽器): __ 40 __ 時間程度かかると想定
- 作業 F(Processing): __ 15 __ 時間程度かかると想定
- 作業 G(統合・テスト): __ 20 __ 時間程度かかると想定

見積もりは正確である必要はないが、「思ったより時間がかかる可能性」を考慮する。

6. 並行作業と依存関係

以下の作業は並行して進めることができる。

- ハードウェア実装とソフトウェア実装。
- 楽器システムの開発と Processing による音生成システムの構築

一方で、以下の作業は順番に進める必要がある。

- センサー値取得・閾値設定 → 同期フラグ・テンポ変化フラグの生成 → 通信制御 → 各 Arduino での動作

(理由: 前の処理結果が後の処理に影響するため。)

7. ガントチャートへの反映

上記の作業を週単位に割り当て、ガントチャートとして整理する。

- 実装は第__6__週から開始する
- 第__8__週には、システム全体を一度動かす週を設ける
- 最終週の前に、調整・改善のための余裕週を設ける

ガントチャートは固定せず、進捗に応じて見直す。

8. リスクと対策

想定されるリスクとして、以下を挙げる。

- 想定より実装に時間がかかる
- ハードウェアがうまく動作しない
- メンバー間で進捗に差が出る

それぞれに対して、以下の対策を考える。

- 3 年が実装の詳細に書いた仕様書を共有する。
 - 最初のタスクにして、改善の時間を設ける。
 - 終わった人が手伝うことができるタスクを洗い出しておく。
-

9. 進捗確認の方法

毎週、以下の点をチームで確認する。

- 今週予定していた作業は完了したか
- 予定から遅れている作業は何か
- 次週に影響が出そうな問題はあるか

必要に応じて、スケジュールを修正する。

10. 最終確認

この計画について、以下を確認する。

- 実装が後半に集中しすぎている
- 統合・テストの期間が確保されている
- 発表準備が最終週だけになっていない

以上を満たしている場合、この計画を実行に移す。